University of Information Technology and Sciences (UITS)

Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE0613212

Course Title: Data Structures and Algorithms I Lab

Program: B.Sc. in CSE

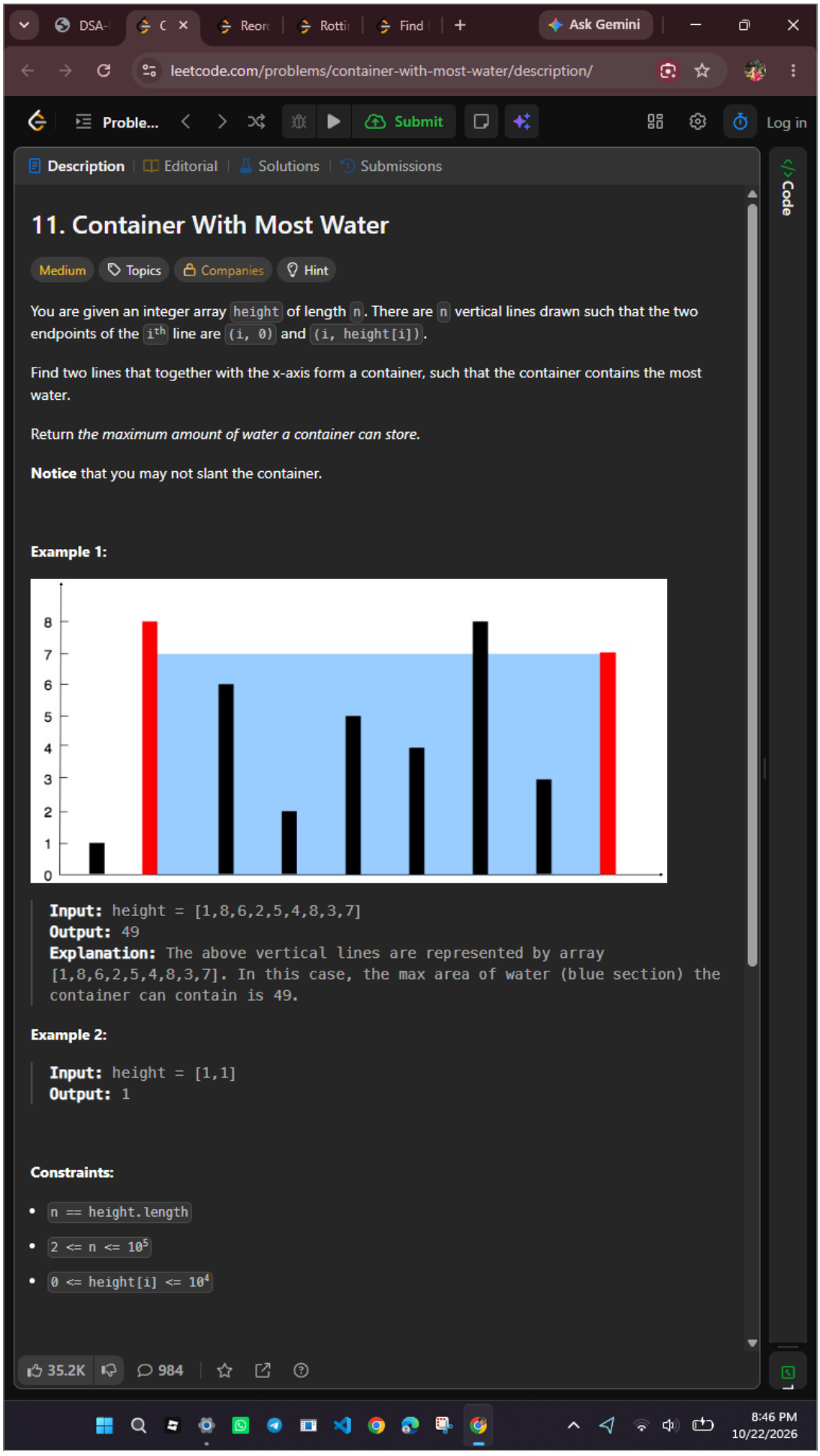
Topics: Arrays, Linked List, Matrix/Grid, Searching

Submitted to	Md. Yousuf Ali
Submitted by	Auntika Barua
Student ID	04325205101018
Batch	58
Section	3A1
Session	Autumn 2026

Problems solved: LeetCode 11, 143, 994, 153 (C++)

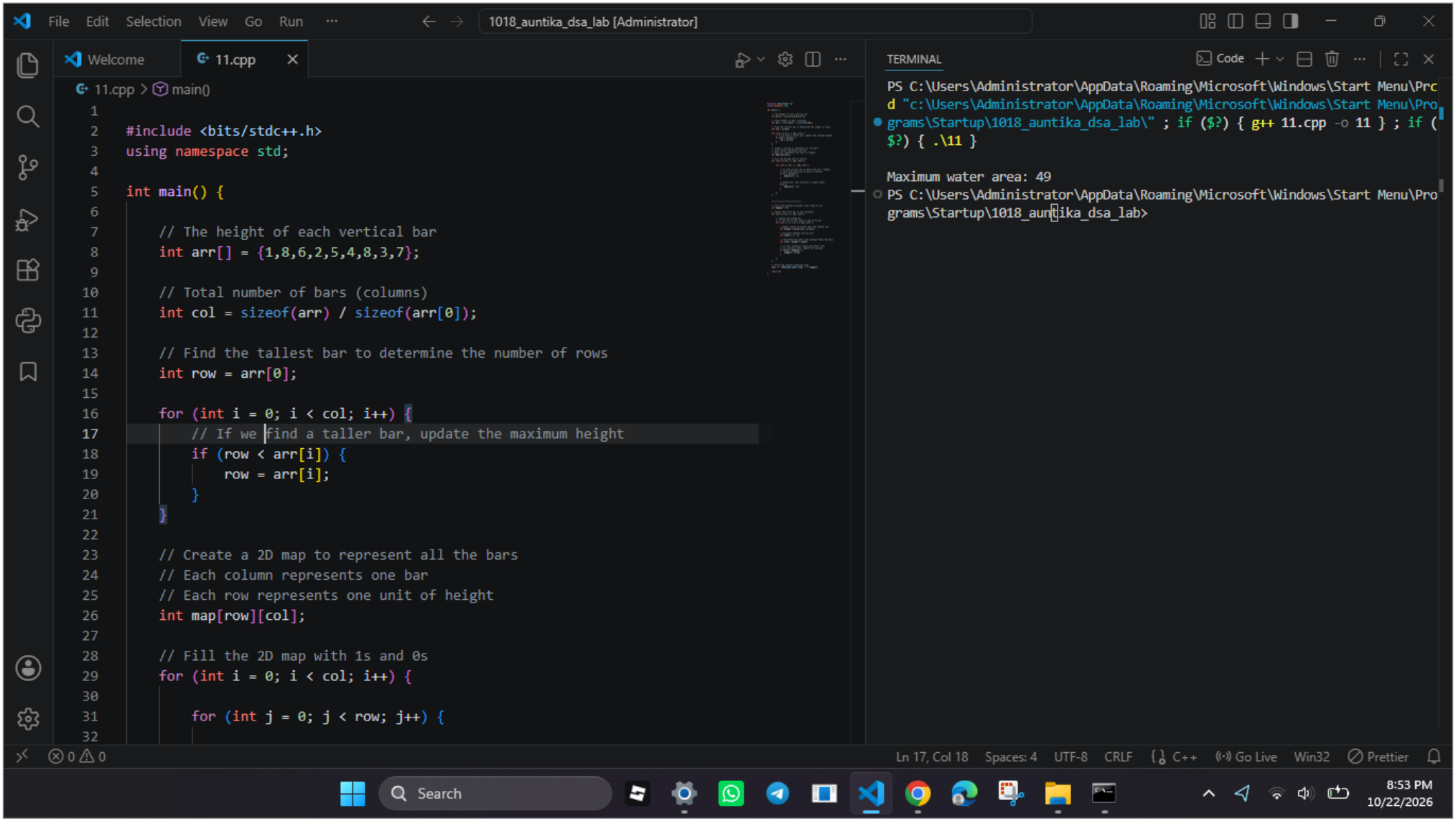
Question 1: LeetCode 11 - Container With Most Water

Figure 1: Problem statement



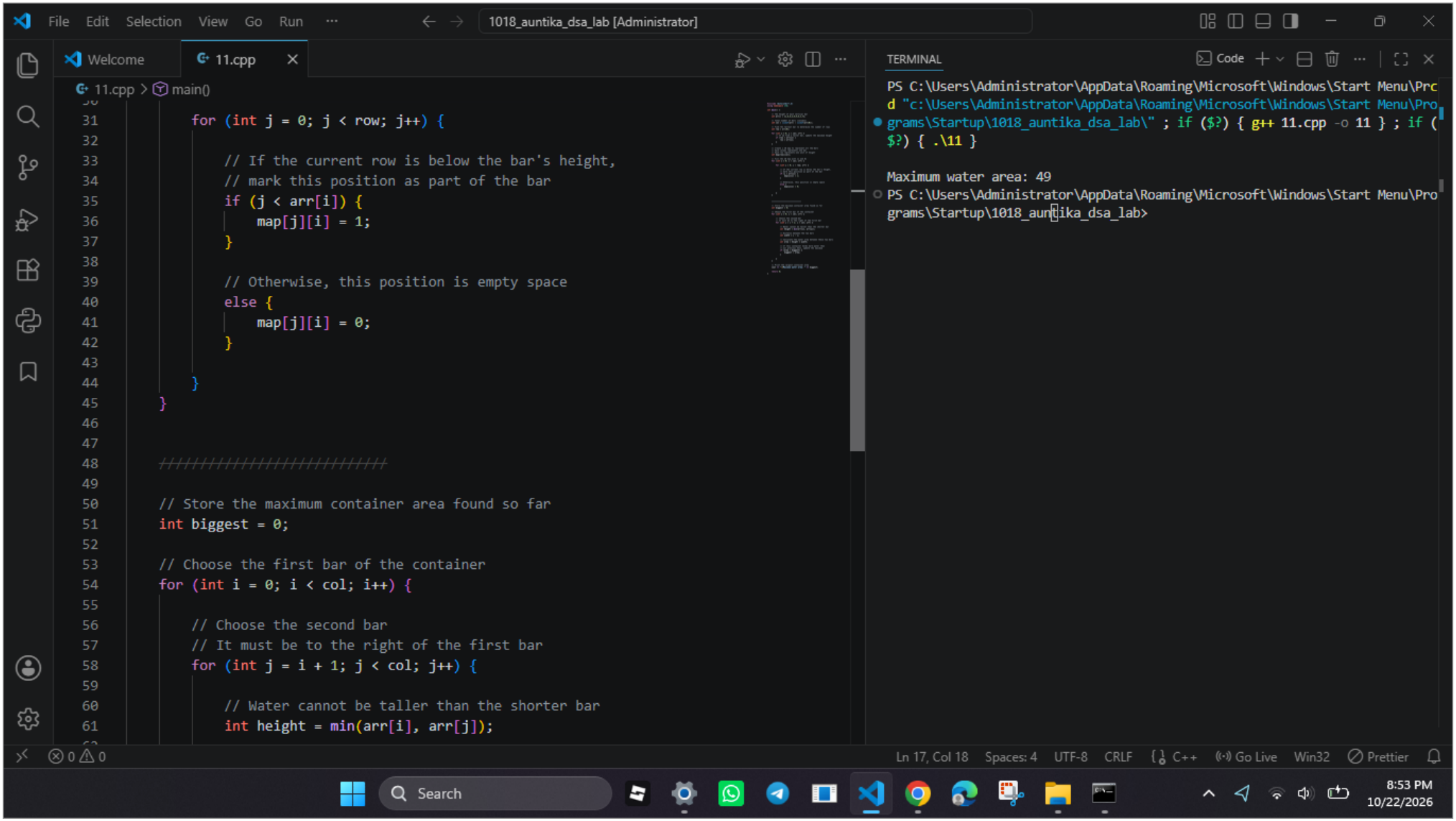
Question 1: LeetCode 11 - Container With Most Water

Figure 2: Code: input array, 2D map construction (part 1)



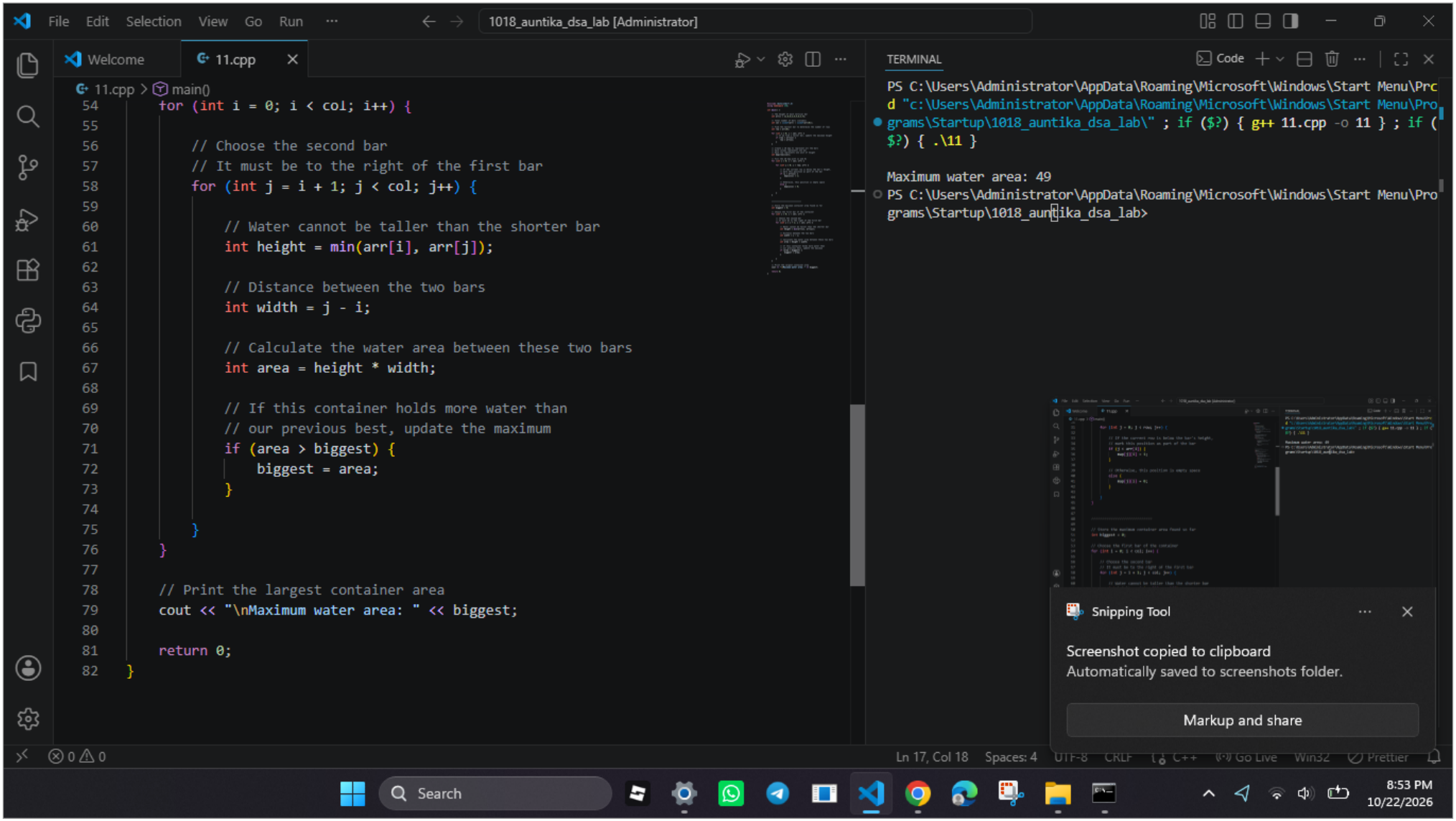
Question 1: LeetCode 11 - Container With Most Water

Figure 3: Code: map construction and container loop (part 2)



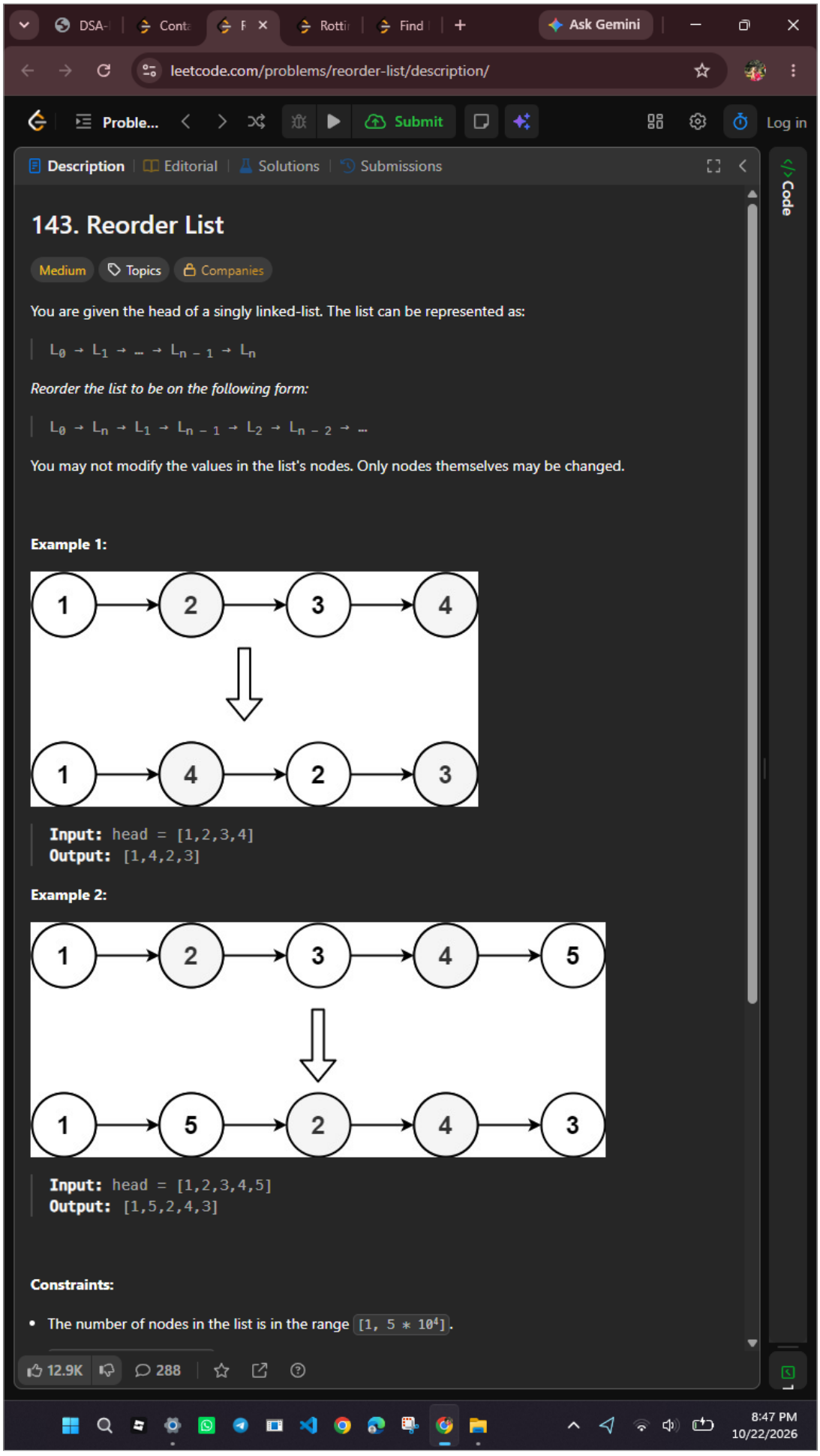
Question 1: LeetCode 11 - Container With Most Water

Figure 4: Code: area calculation, final output (49)



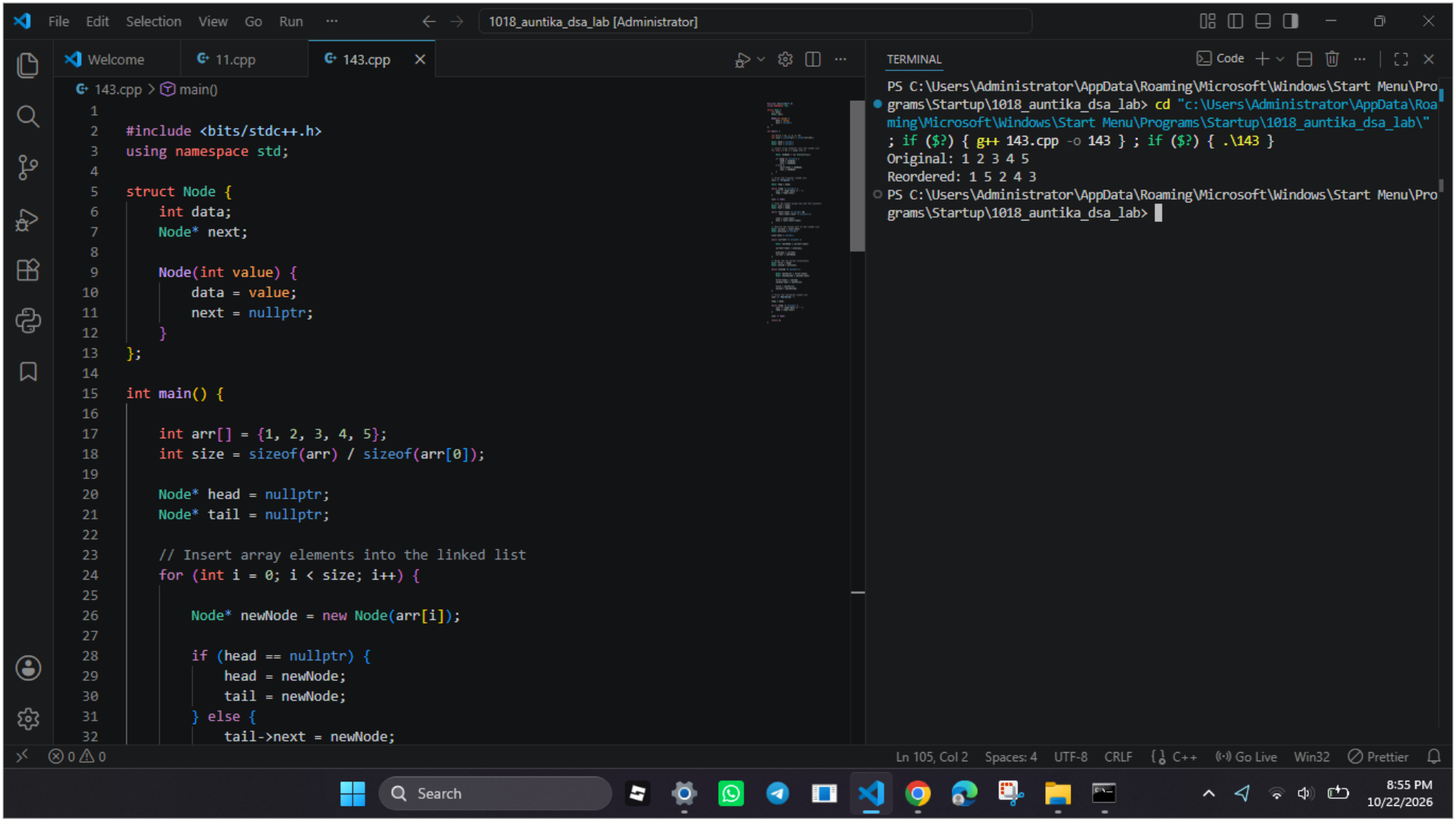
Question 2: LeetCode 143 - Reorder List

Figure 5: Problem statement



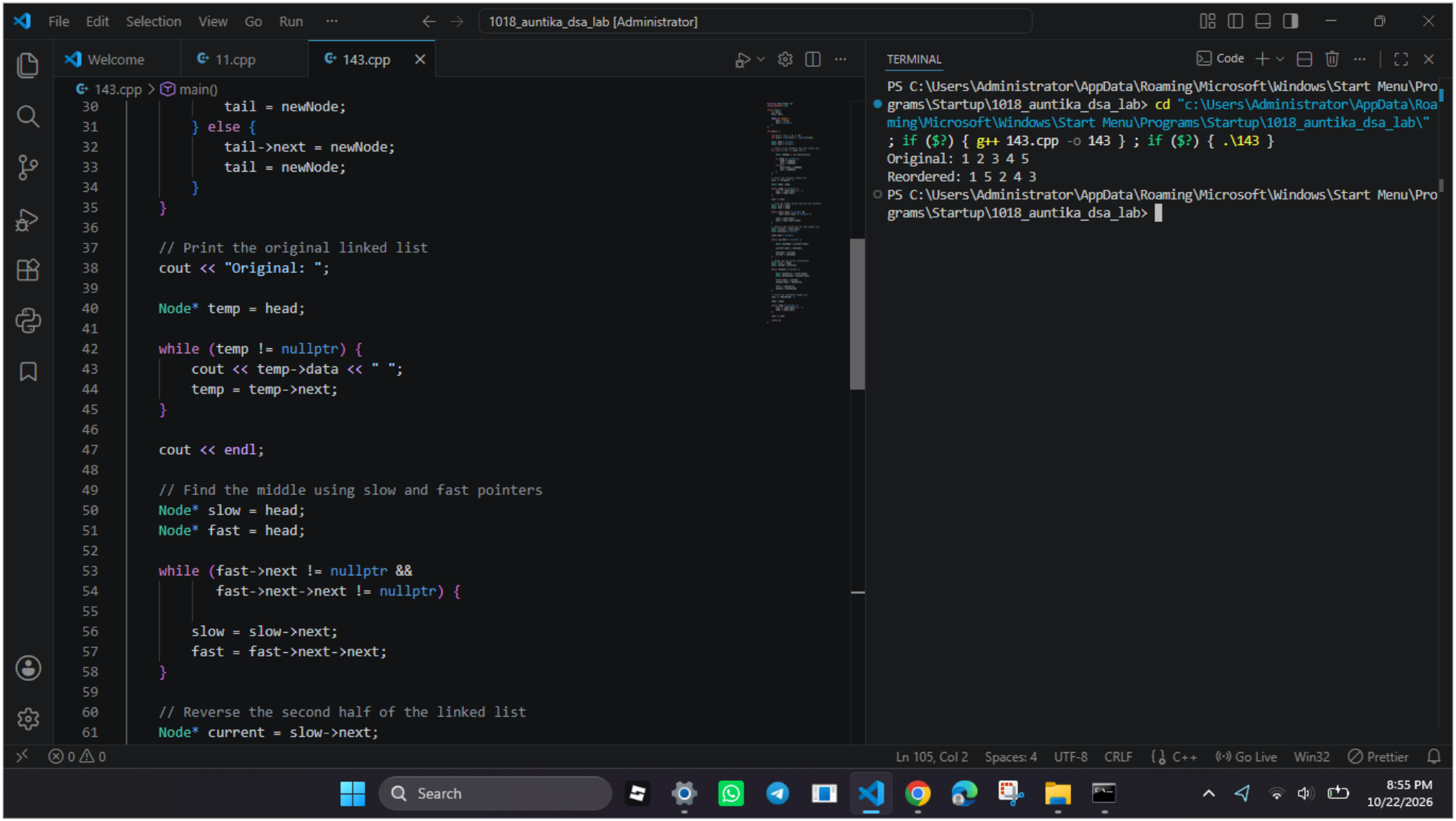
Question 2: LeetCode 143 - Reorder List

Figure 6: Code: node structure, list creation (output shown)



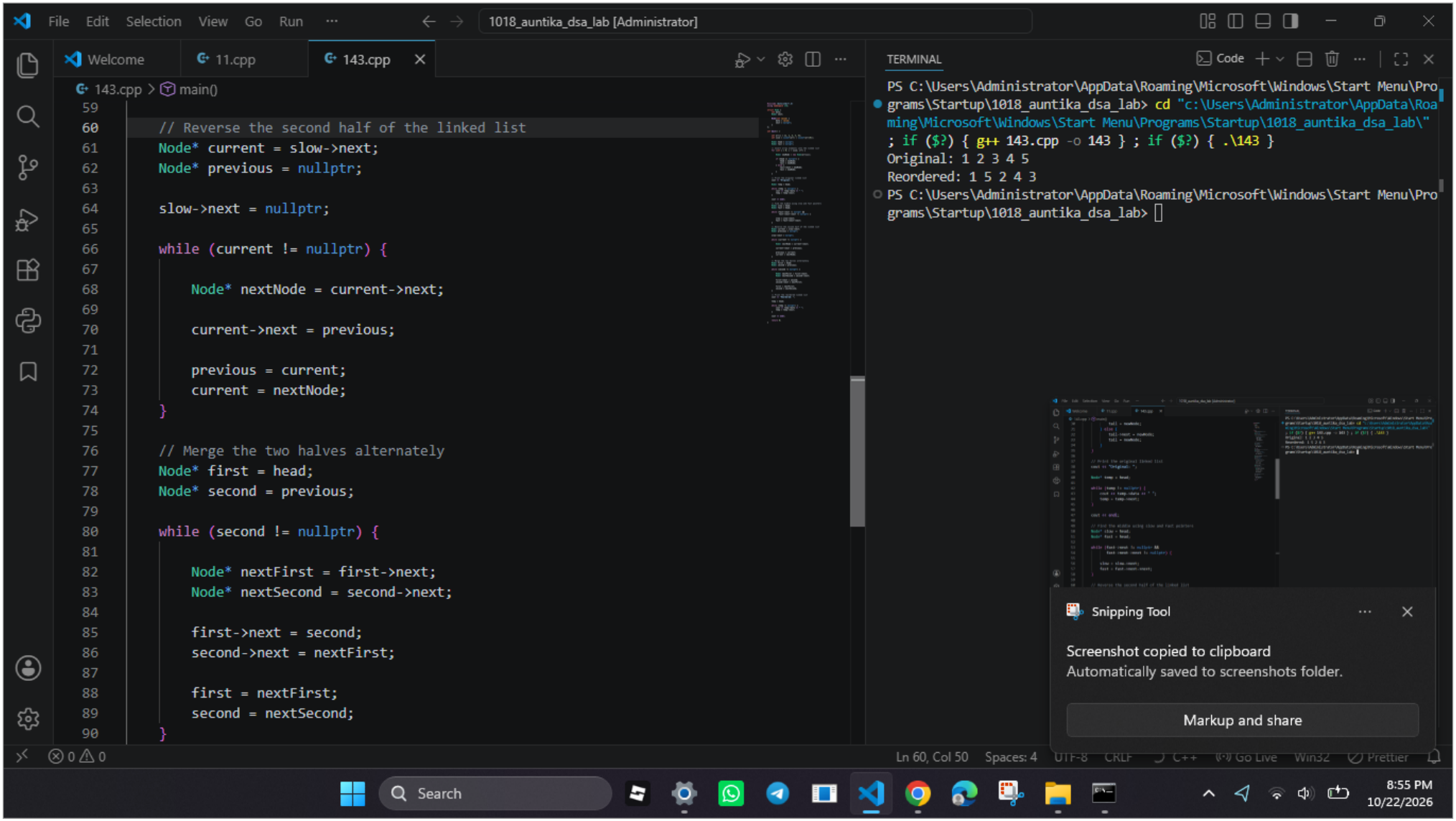
Question 2: LeetCode 143 - Reorder List

Figure 7: Code: print, find middle (slow/fast pointers)



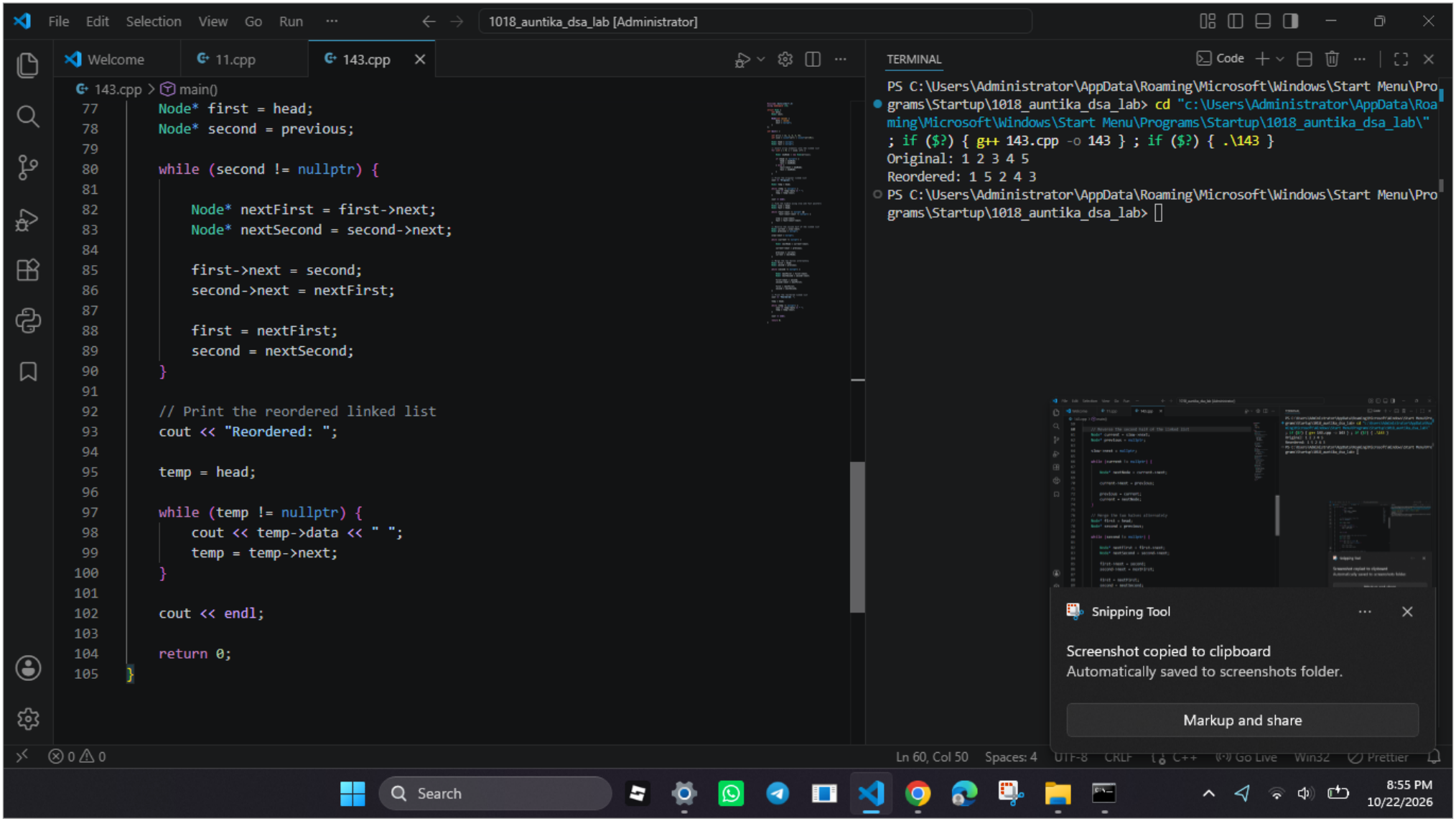
Question 2: LeetCode 143 - Reorder List

Figure 8: Code: reverse second half, alternate merge



Question 2: LeetCode 143 - Reorder List

Figure 9: Code: merge loop, print reordered list



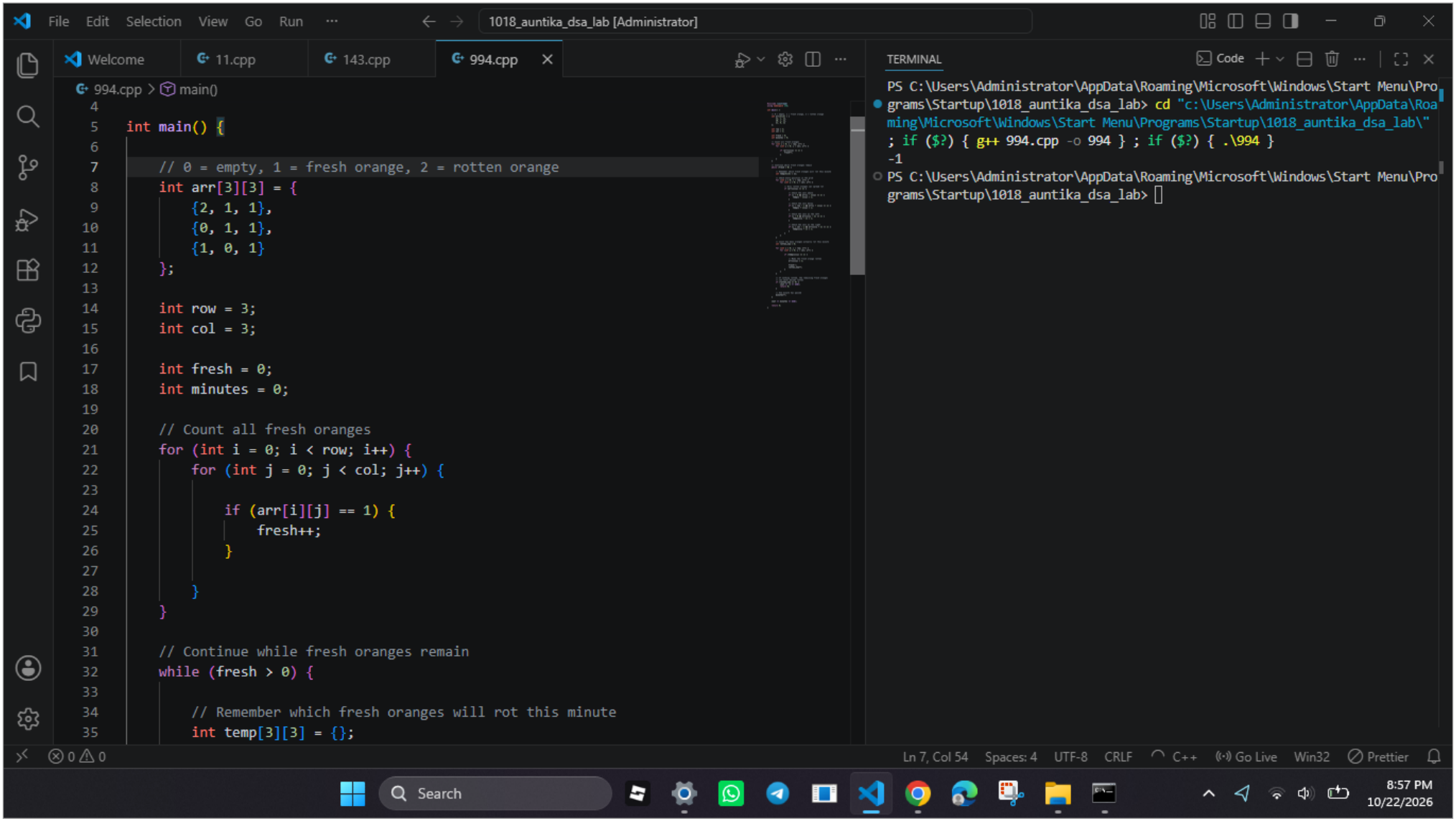
Question 3: LeetCode 994 - Rotting Oranges

Figure 10: Problem statement



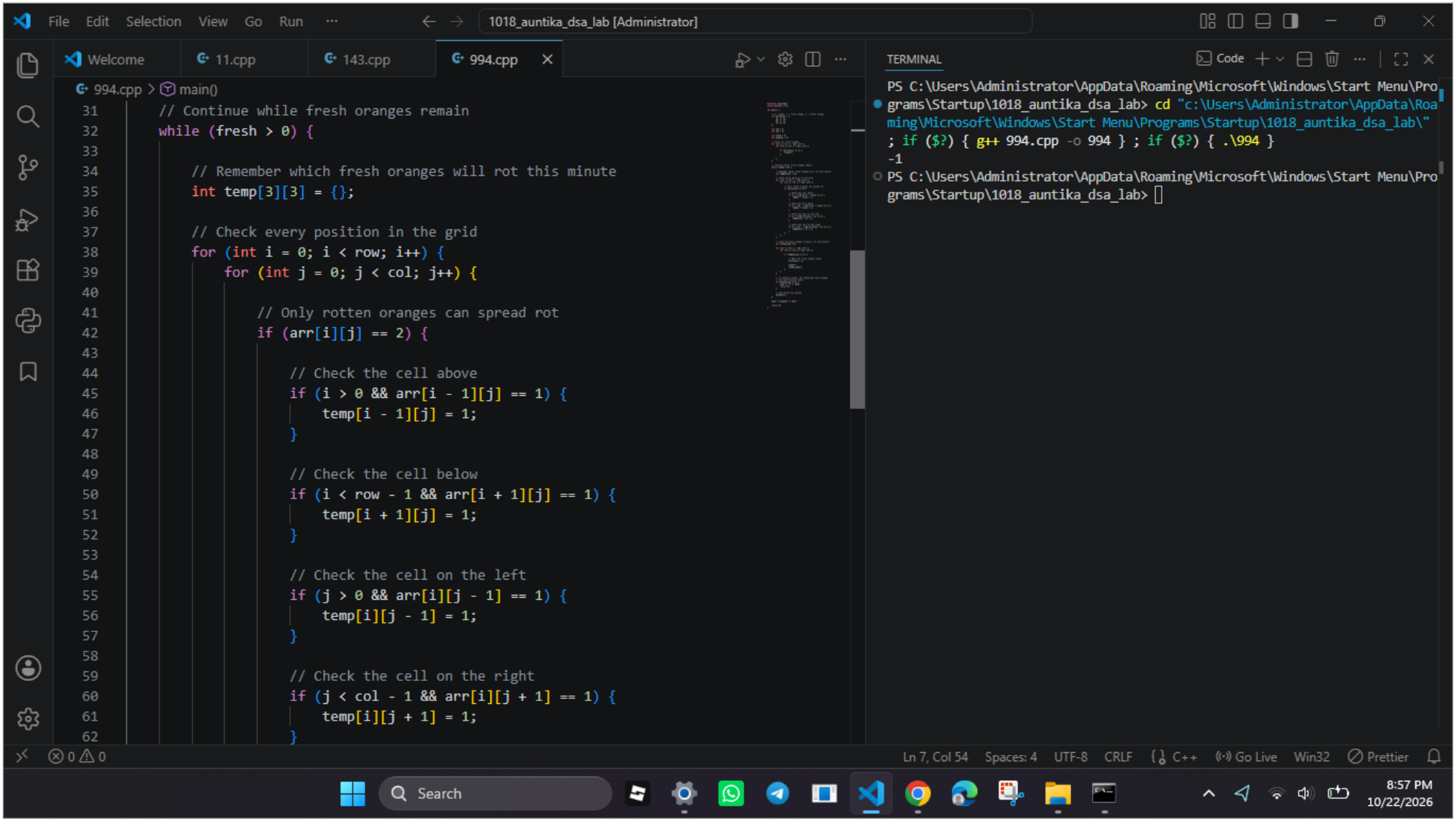
Question 3: LeetCode 994 - Rotting Oranges

Figure 11: Code: grid, fresh orange count (output -1)



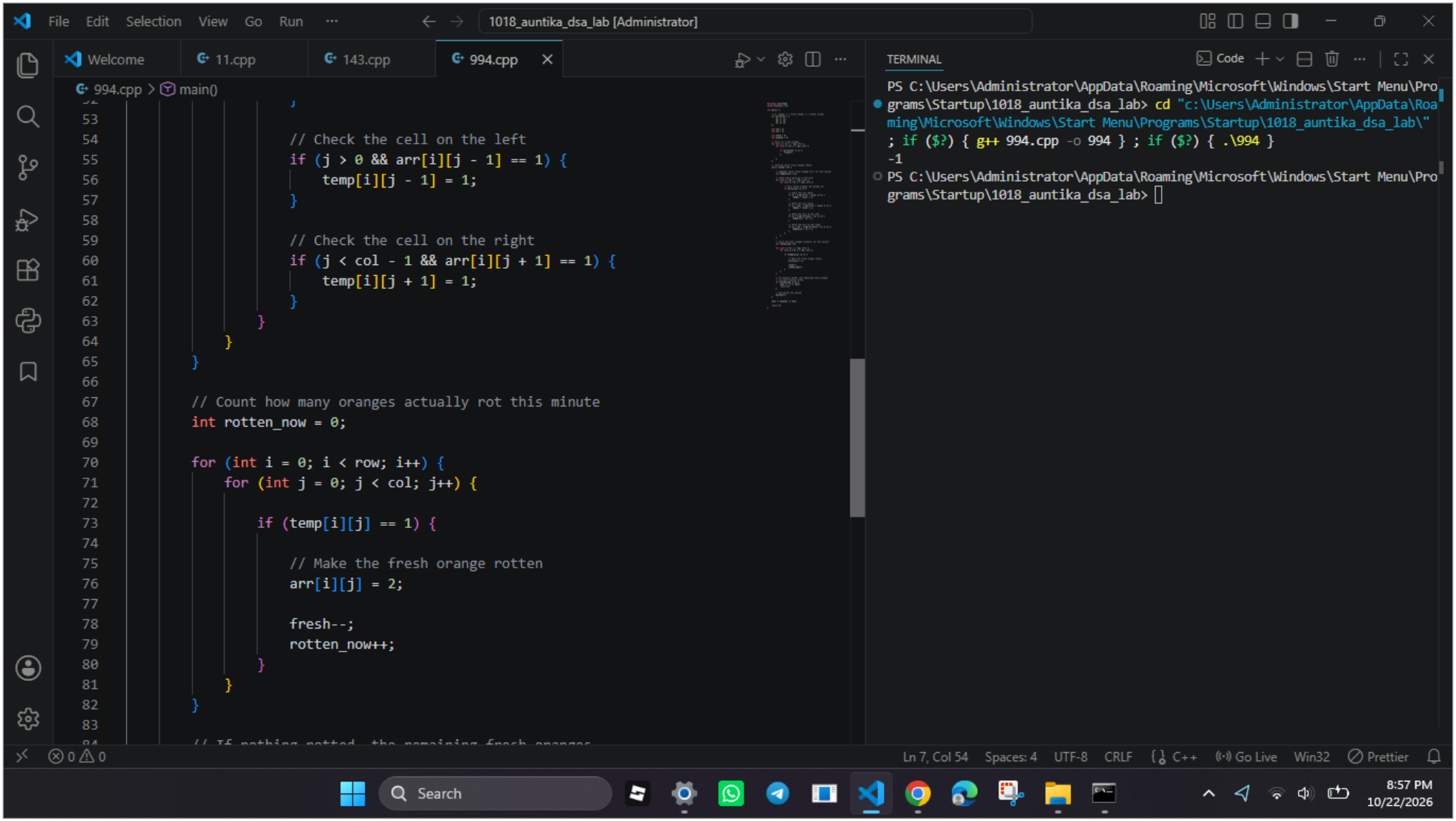
Question 3: LeetCode 994 - Rotting Oranges

Figure 12: Code: rot spreading in four directions



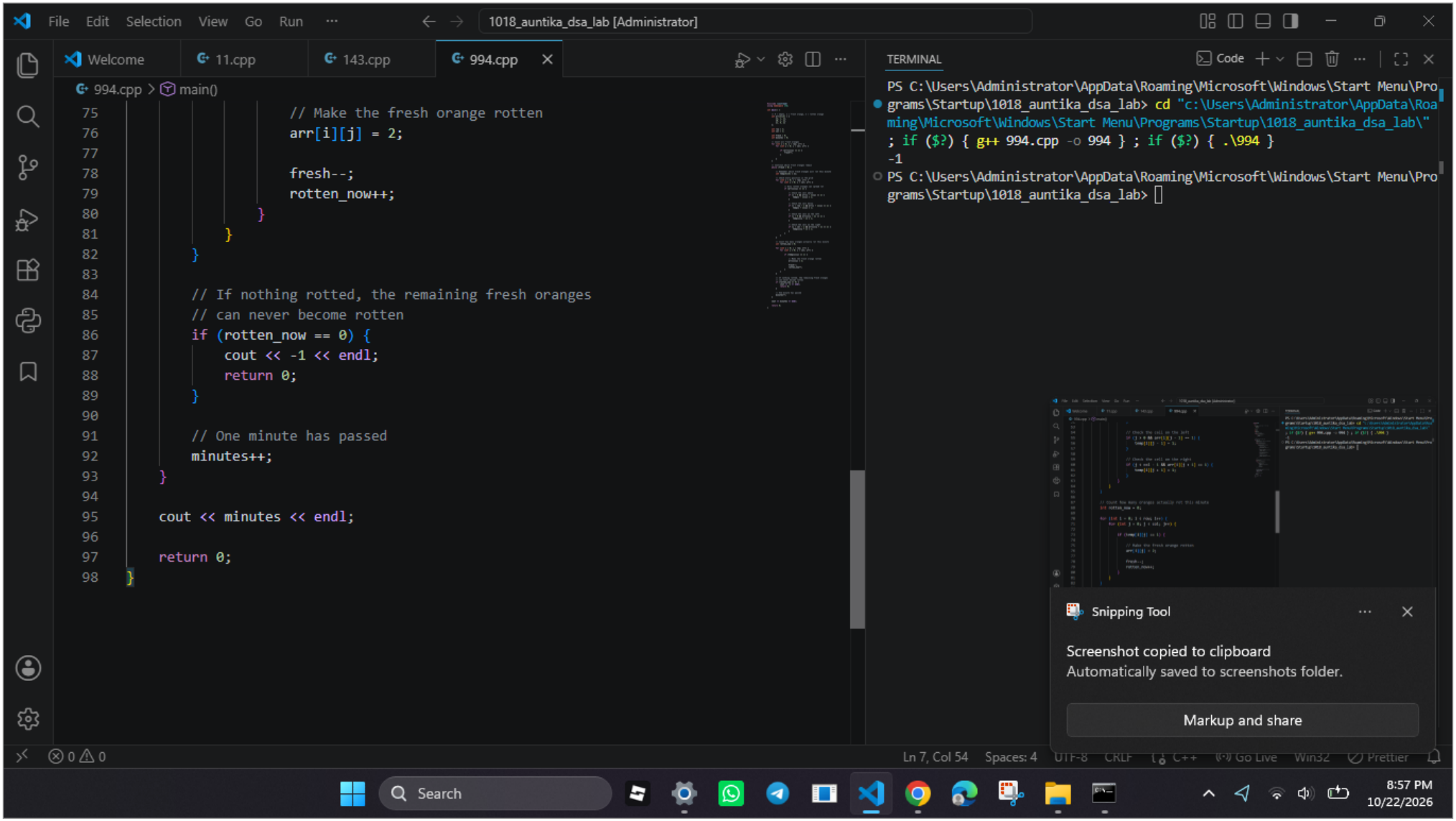
Question 3: LeetCode 994 - Rotting Oranges

Figure 13: Code: apply rot, count rotted this minute



Question 3: LeetCode 994 - Rotting Oranges

Figure 14: Code: impossible check, minutes, output



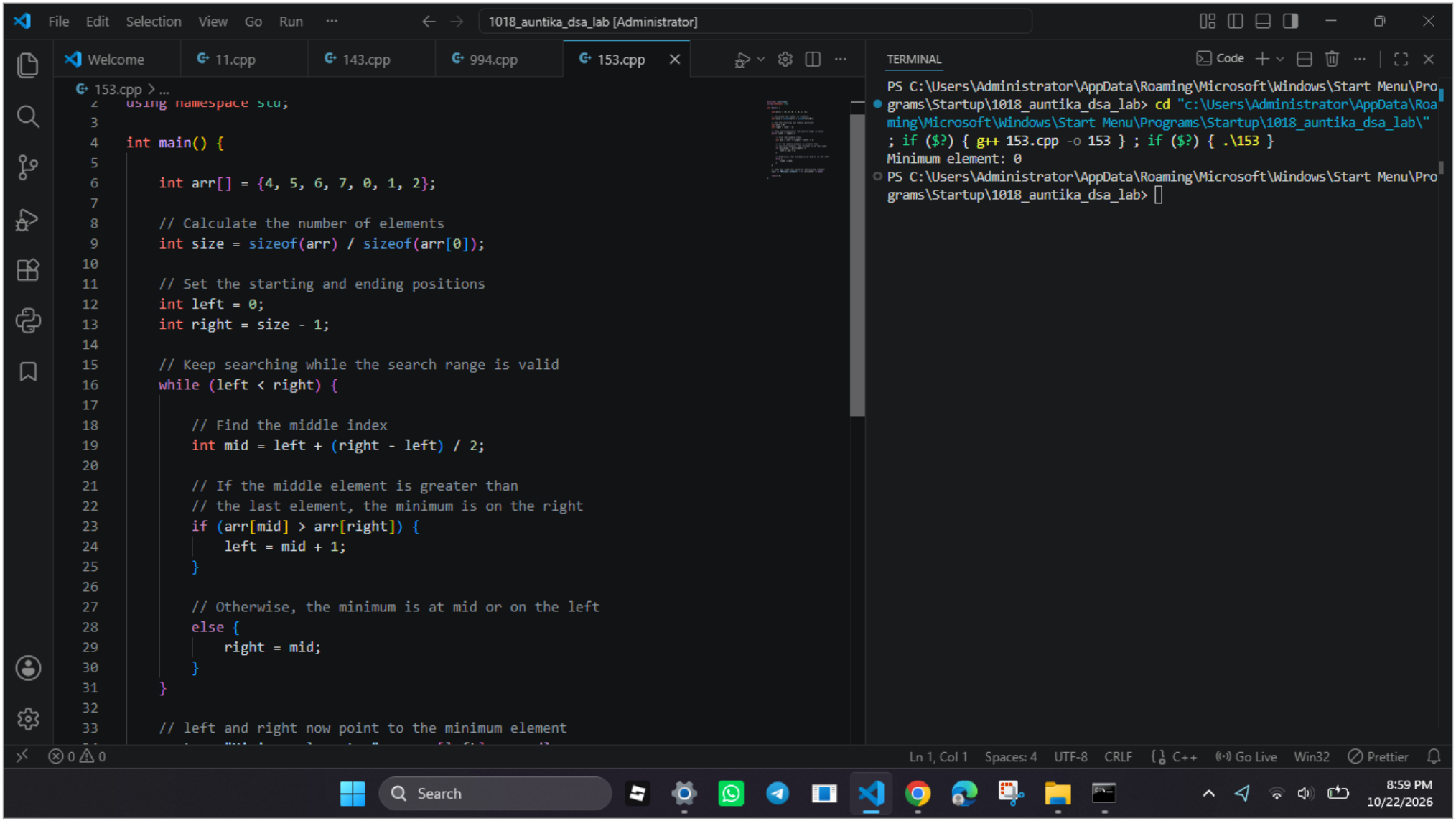
Question 4: LeetCode 153 - Find Minimum in Rotated Sorted Array

Figure 15: Problem statement



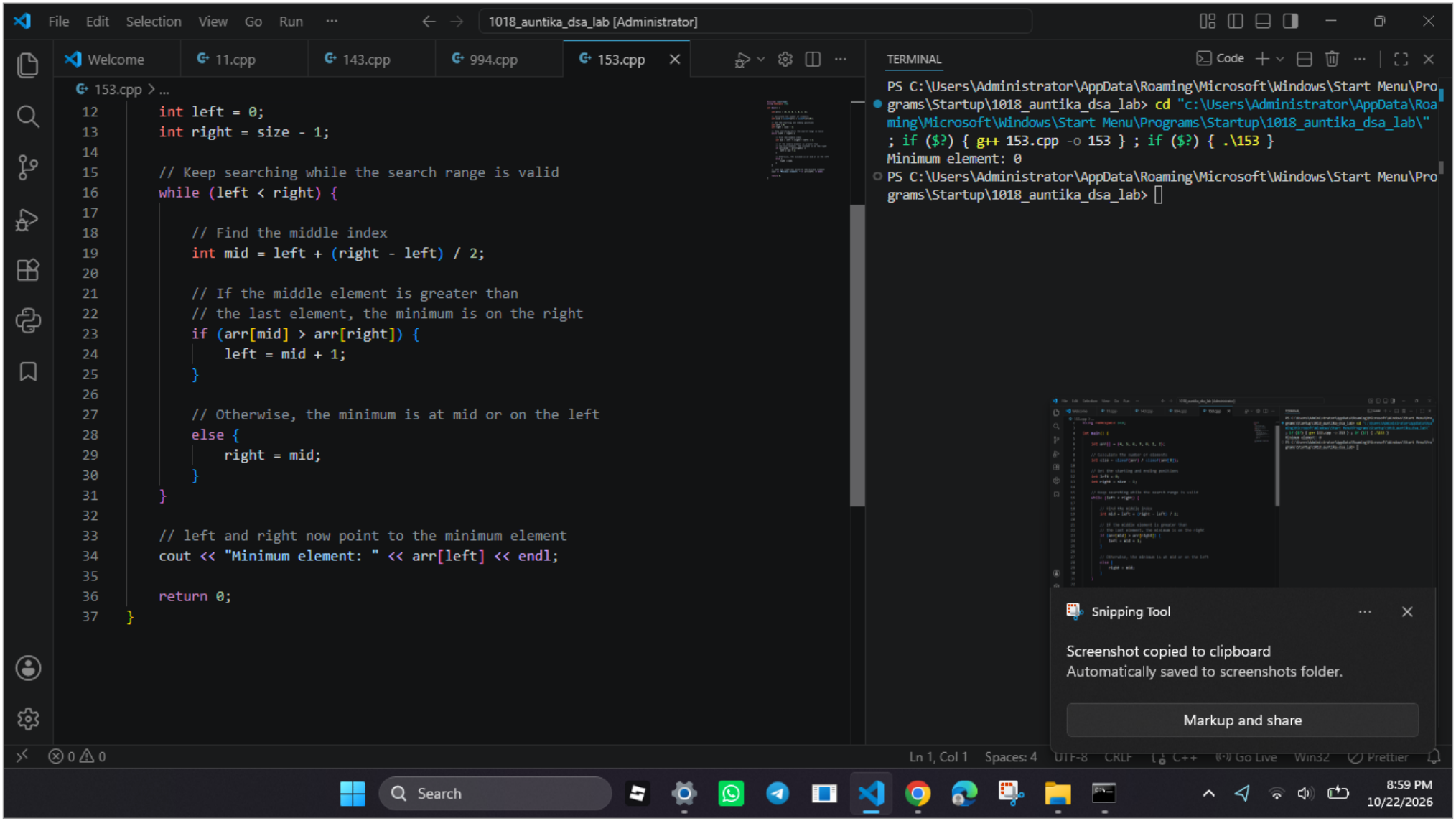
Question 4: LeetCode 153 - Find Minimum in Rotated Sorted Array

Figure 16: Code: binary search setup and loop (output 0)



Question 4: LeetCode 153 - Find Minimum in Rotated Sorted Array

Figure 17: Code: binary search loop and result print



Justification: Relation of the Four Problems to COs and POs

Course: CSE0613212, Data Structures and Algorithms I Lab

1. CO-PO mapping of the course (from the course outline)

CO	Course Outcome	Mapped PO
CO1	Demonstrate knowledge of fundamental data structures and algorithms.	PO(a) Engineering Knowledge
CO2	Implement fundamental data structures and algorithms using programming tools to solve specified computational problems.	PO(e) Modern Tool Usage
CO3	Analyze problem requirements and justify the selection of appropriate data structures and algorithms.	PO(b) Problem Analysis
CO4	Design and develop algorithmic solutions for real-world problems using appropriate fundamental data structures and algorithms.	PO(c) Design/Development of Solutions

2. Problem-wise justification

Q1. LeetCode 11: Container With Most Water	
Course topic	Array operations and matrix (2D array) operations; brute-force searching of pairs (Weeks 2-4).
What was implemented	The heights are stored in an array and converted into a 2D map of 1s and 0s to model the bars. Every pair of bars (i, j) is checked, the water level is limited by the shorter bar, area = min(height) x (j - i), and the maximum is tracked. Output for the sample input: 49, matching the expected result.
CO relation	**CO2**: arrays and a 2D array are implemented in C++ to solve a specified problem. **CO3**: the requirement (water cannot rise above the shorter bar) is analyzed and exhaustive pair checking is chosen and justified as a correct approach.
PO relation	**PO(e)**: C++ and the VS Code/g++ toolchain are used to implement and test the solution. **PO(b)**: the problem is broken down and analyzed into height, width and area.

Q2. LeetCode 143: Reorder List	
Course topic	Linked List I and II: dynamic memory allocation, traversal, pointer manipulation (Weeks 5-6).
What was implemented	A singly linked list is built with dynamically allocated nodes. The middle is found with slow and fast pointers, the second half is reversed in place, and the two halves are merged alternately by changing only the next pointers. Output: 1 2 3 4 5 becomes 1 5 2 4 3.
CO relation	**CO1**: demonstrates knowledge of the node structure, pointers and linked list operations. **CO2**: the list operations (create, traverse, reverse, merge) are implemented to solve a defined problem.
PO relation	**PO(a)**: applies core computing knowledge of pointers, memory and linked data structures. **PO(e)**: the solution is coded, compiled, run and verified with programming tools.

Q3. LeetCode 994: Rotting Oranges	
Course topic	Matrix operations (Week 3) and grid/graph traversal ideas related to BFS (Week 12).
What was implemented	A 2D grid is scanned every minute; rotten cells infect fresh neighbours in four directions using a temporary grid so that rotting happens simultaneously. The fresh count and the elapsed minutes are updated, and -1 is returned when no orange rots in a minute while fresh ones remain. Tested on the grid [[2,1,1],[0,1,1],[1,0,1]]; output: -1, which is correct.
CO relation	**CO2**: 2D arrays and loops are implemented to solve a specified problem. **CO4**: a step-by-step algorithm is designed for a real-world style spreading process, including the impossible-case condition.
PO relation	**PO(e)**: implementation and testing with programming tools. **PO(c)**: a solution to an open-ended problem is designed with the required constraints and edge cases.

Q4. LeetCode 153: Find Minimum in Rotated Sorted Array	
Course topic	Searching algorithms: binary search (Week 4), with algorithm complexity analysis (Week 1).
What was implemented	Binary search with left, right and mid is applied to the rotated array: if arr[mid] > arr[right] the minimum lies right of mid, otherwise at mid or to its left. This runs in O(log n), as the problem requires. Output for [4,5,6,7,0,1,2]: 0.
CO relation	**CO1**: demonstrates knowledge of the binary search algorithm. **CO2**: the algorithm is implemented in C++. **CO3**: binary search is selected over linear scan and justified by the O(log n) requirement.
PO relation	**PO(a)**: algorithm and complexity knowledge. **PO(e)**: implementation with programming tools. **PO(b)**: the rotated-array property is analyzed to decide which half to discard.

3. Summary matrix

Problem	CO1	CO2	CO3	CO4	PO(a)	PO(b)	PO(c)	PO(e)
Q1 LC 11	-	X	X	-	-	X	-	X
Q2 LC 143	X	X	-	-	X	-	-	X
Q3 LC 994	-	X	-	X	-	-	X	X
Q4 LC 153	X	X	X	-	X	X	-	X

Together, the four problems cover all four course outcomes (CO1 to CO4) and the mapped program outcomes PO(a), PO(b), PO(c) and PO(e).